

Machine Learning Intro

Teach Computers to Learn From Data

■ ML FOUNDATIONS

■ LEARNING OUTCOME

By the end of this PDF you will understand the ML landscape, train your first classification and regression models using scikit-learn, and know how to prevent overfitting with regularization and cross-validation.

■ Skills You Will Gain

- Understand supervised vs unsupervised vs reinforcement learning
- Prepare data using train/test split and cross-validation
- Train linear regression, logistic regression, and decision trees
- Train ensemble models: Random Forest and Gradient Boosting
- Apply regularization to prevent overfitting
- Understand the bias-variance tradeoff

■ ML is not magic — it's statistics + optimization on data. Every model you use is just a mathematical function trying to find patterns. Understanding this makes you a better practitioner.

SECTION 1

ML Landscape

Type	Description	Examples
Supervised Learning	Learn from labeled data (input → correct output)	Classification: spam/not-spam Regression: house price prediction
Unsupervised Learning	Find patterns in unlabeled data	Clustering: customer segments Dimensionality: PCA
Semi-supervised	Mix of labeled and unlabeled data	Text classification with few labels
Reinforcement Learning	Agent learns by reward/punishment	Game playing, robotics, recommendation
Self-supervised	Model creates its own labels from data	GPT, BERT language models

SECTION 2

The ML Pipeline

1	Problem Definition Classification? Regression? Clustering? What is the target variable?
2	Data Collection Internal databases, APIs, web scraping, public datasets
3	EDA & Cleaning Understand, clean, and fix the data (Modules 3+4)
4	Feature Engineering Create and select the best features (Module 5)
5	Model Selection Choose algorithm family based on problem type and data size
6	Train & Tune Fit model on training data, tune hyperparameters with cross-validation
7	Evaluate Test on held-out data — Module 9 covers this in depth
8	Deploy Serve predictions via API (Flask/FastAPI) or batch job

SECTION 3

sklearn API Pattern — Learn Once, Use Everywhere

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Every sklearn model follows this exact same pattern:
# 1. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 2. Scale features
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
# 3. Instantiate model
model = LogisticRegression(max_iter=1000)
# 4. Train (fit) model
model.fit(X_train_sc, y_train)
# 5. Predict
y_pred = model.predict(X_test_sc)
# 6. Evaluate
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
```

SECTION 4

Core Algorithms

Algorithm	Type	Best When	sklearn Class
Linear Regression	Regression	Numeric target, linear relationships	LinearRegression()
Logistic Regression	Classification	Binary target, interpretability needed	LogisticRegression()
Decision Tree	Both	Non-linear, interpretable rules	DecisionTreeClassifier()
Random Forest	Both	Tabular data, robust to noise	RandomForestClassifier()
Gradient Boosting	Both	Best accuracy on tabular data	GradientBoostingClassifier()
XGBoost	Both	Kaggle competitions, production	XGBClassifier()
K-Nearest Neighbors	Both	Small datasets, non-parametric	KNeighborsClassifier()
SVM	Classification	High-dimensional, small datasets	SVC()
K-Means	Clustering	Customer segmentation	KMeans()
PCA	Dim. Reduction	Visualization, reduce noise	PCA()

SECTION 5

Bias-Variance Tradeoff & Regularization

The Fundamental ML Problem

High Bias (Underfitting): Model is too simple → misses patterns → bad on both train AND test
High Variance (Overfitting): Model memorizes training data → great on train, bad on test
Goal: Find the sweet spot — complex enough to learn, simple enough to generalize

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
# Ridge (L2): penalizes large coefficients — shrinks towards zero but never exactly zero
ridge = Ridge(alpha=1.0)
# alpha controls regularization strength
# Lasso (L1): some coefficients become exactly zero — automatic feature selection
lasso = Lasso(alpha=0.1)
# ElasticNet: combines L1 + L2
en = ElasticNet(alpha=0.1, l1_ratio=0.5)

# Cross-validation to find best alpha from sklearn.linear_model
ridge_cv = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100])
ridge_cv.fit(X_train, y_train)
print(f'Best alpha: {ridge_cv.alpha_}')
```

SECTION 6

Cross-Validation

```
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold # K-Fold
Cross-Validation (K=5) kf = KFold(n_splits=5, shuffle=True, random_state=42) scores =
cross_val_score(model, X, y, cv=kf, scoring='accuracy') print(f'CV Accuracy: {scores.mean():.4f} ±
{scores.std():.4f}') # StratifiedKFold – preserves class balance (use for classification) skf =
StratifiedKFold(n_splits=5) scores = cross_val_score(model, X, y, cv=skf, scoring='roc_auc') #
GridSearchCV – find best hyperparameters from sklearn.model_selection import GridSearchCV
param_grid = {'n_estimators':[100,200], 'max_depth':[3,5,None]} grid =
GridSearchCV(RandomForestClassifier(), param_grid, cv=5, scoring='f1') grid.fit(X_train, y_train)
print(f'Best params: {grid.best_params_}')
```

■ PRACTICE QUESTIONS

Q1. What is the difference between classification and regression?

Answer: Classification predicts a discrete category (spam/not-spam, disease/no-disease). Regression predicts a continuous number (house price, temperature).

Q2. Why do we need a test set that the model never sees during training?

Answer: To measure how well the model generalizes to new, unseen data — its real-world performance. Training accuracy alone is misleading (model may have memorized).

Q3. What is overfitting and how do you detect it?

Answer: Model memorizes training data — high training accuracy but low test accuracy. Detect by large gap between train and validation performance.

Q4. What does Ridge regression do differently from standard linear regression?

Answer: Ridge adds a penalty for large coefficients (L2 regularization), shrinking them towards zero. This reduces overfitting but keeps all features.

Q5. When would you choose Random Forest over Logistic Regression?

Answer: Non-linear relationships in data, need for robustness to noise, tabular data with mixed feature types. Logistic regression is better when interpretability is critical.

■ Machine Learning Cheat Sheet	
<code>train_test_split(X,y,test_size=.2)</code>	Split data 80/20
<code>model.fit(X_train,y_train)</code>	Train the model
<code>model.predict(X_test)</code>	Make predictions
<code>cross_val_score(model,X,y,cv=5)</code>	5-fold cross-validation
<code>LinearRegression()</code>	Regression: linear
<code>LogisticRegression()</code>	Binary classification
<code>RandomForestClassifier()</code>	Ensemble: bagging
<code>GradientBoostingClassifier()</code>	Ensemble: boosting
<code>Ridge(alpha=1.0)</code>	L2 regularization
<code>Lasso(alpha=0.1)</code>	L1 + feature selection
<code>GridSearchCV(model,params,cv)</code>	Hyperparameter tuning
<code>KMeans(n_clusters=k)</code>	Unsupervised clustering